

Agent 能力接口设计

更新日期：2026-06-29

本文定义供 AI agent 访问的受控能力接口。目标是让 agent 可以协助筛选市场、生成策略草案、解释风险、提交审批和跟踪执行结果，但不能绕过权限、预算、人工确认或审计系统。

如果目标是让 agent 直接接入和调用系统，请先阅读：[Agent 接入手册](#)。本文更偏接口设计和权限模型说明。

核心原则：

- agent 访问的是受控能力，不是任意 shell。
- agent 可以创建和修改草案，但不能批准自己的策略。
- 金额、仓位、滑点、市场范围、审批阈值必须由后端强制检查。
- 首页需要实时显示 agent 正在做什么，以及哪些策略等待人工确认。
- 已提交、已批准、已运行的策略不做物理删除，改用取消、暂停、归档等状态。

1. 设计边界

1.1 角色

角色	能做什么	不能做什么
Agent	读市场、读有限账户视图、生成策略草案、修改草案、运行风险检查、运行模拟、提交审批、提出调整建议	批准策略、提高自己的权限、修改资金上限、绕过风控、直接写密钥、直接执行未批准订单
人类用户	审批、拒绝、要求修改、配置资金、暂停/恢复/终止策略、管理权限	不应绕过审计直接改数据库
执行器	按已批准策略生成执行计划、执行订单、记录结果、执行前重新风控	不接受 agent 手写的裸订单
设置端	管理 agent 权限、预算、白名单、审批阈值、紧急停止	不参与策略推理

1.2 与现有 mode / state 的关系

当前系统已有：

```
mode   = Stop / Virtual / Real
state  = strategy_state.namespace = machine 的策略状态机
```

Agent 审批流不要覆盖这两个概念。建议新增一个策略生命周期字段，暂命名为：

```
lifecycle_state
```

前端可以把它显示为 AI State / Approval State / 确认状态 ，不要和 mode 混在一起。

示例：

```
mode = Stop
state = auto
lifecycle_state = WAITING_HUMAN_CONFIRM
```

含义是：策略暂不运行，策略内部状态为 auto，但 AI 已经提交，正在等待人工确认。

2. 状态设计

2.1 策略生命周期状态

推荐状态：

状态	中文显示	含义
AI_DRAFTING	AI 编写中	agent 正在生成或修改策略草案
AI_PROPOSED	AI 已提案	agent 已形成可审阅草案，但尚未提交审批
WAITING_HUMAN_CONFIRM	待人工确认	agent 已提交审批单，等待人类批准/拒绝/要求修改
HUMAN_REVISION_REQUESTED	人工要求修改	人类要求 agent 修改后再提交
HUMAN_APPROVED	已人工批准	审批通过，已冻结策略快照

状态	中文显示	含义
HUMAN_REJECTED	已拒绝	人类拒绝该策略
RISK_BLOCKED	风控阻断	风控检查未通过，不能进入审批或执行
ACTIVE	运行中	策略已进入 Virtual 或 Real 执行流程
PAUSED	已暂停	人类或系统暂停策略
CANCELLED	已取消	人类取消策略，不再执行
ARCHIVED	已归档	历史策略，只读保留
EXPIRED	已过期	审批单或策略过期，需重新提交

最关键状态是：

WAITING_HUMAN_CONFIRM

它表达“AI 已设置/提交，等待人类确认”的意思，比 AI_SET_WAIT_MAN_CONFIRM 更短、更稳定，也适合前端展示为“待人工确认”。

2.2 状态流

AI_DRAFTING
-> AI_PROPOSED
-> WAITING_HUMAN_CONFIRM
-> HUMAN_APPROVED
-> ACTIVE
-> PAUSED / CANCELLED / ARCHIVED

WAITING_HUMAN_CONFIRM
-> HUMAN_REJECTED
-> HUMAN_REVISION_REQUESTED
-> AI_DRAFTING

AI_PROPOSED / WAITING_HUMAN_CONFIRM
-> RISK_BLOCKED

WAITING_HUMAN_CONFIRM
-> EXPIRED

强制规则：

`created_by_agent = true` 的策略，不能由同一个 `agent` approve。

后端必须检查，不只依赖前端隐藏按钮。

3. 统一接口约定

3.1 能力命名

使用 `domain.action` 命名：

```
market.list
market.get
strategy.draft.create
strategy.draft.update
strategy.submit
risk.check
approval.status
execution.plan
audit.list
```

CLI 形式：

```
agentctl strategy.draft.create --input markets.json --json
```

REST 形式：

```
POST /api/agent/strategy-drafts
```

后续如果要接 MCP，也可以把同一批能力包装成 MCP tools。

3.2 统一返回格式

所有接口默认返回 JSON：

```
{
  "ok": true,
  "data": {},
  "error": null,
  "meta": {
    "request_id": "req...",
    "actor_id": "agent_strategy_assistant",
    "capability": "strategy.draft.create",
    "timestamp": "2026-06-22T00:00:00Z"
  }
}
```

错误返回：

```
{
  "ok": false,
  "data": null,
  "error": {
    "code": "SELF_APPROVAL_FORBIDDEN",
    "message": "Agent cannot approve its own strategy.",
    "details": {
      "agent_id": "agent_strategy_assistant",
      "draft_id": "draft_123"
    }
  },
  "meta": {
    "request_id": "req..."
  }
}
```

3.3 写操作要求

所有写操作必须带：

```
actor_id
idempotency_key
reason
```

高风险写操作还需要：

```
dry_run / apply
approval_id
policy_version
```

4. 能力接口

4.1 能力发现

用途：让 agent 知道自己当前能做什么，以及限制是什么。

CLI:

```
agentctl capabilities
agentctl capability.describe --name strategy.draft.create
```

REST:

```
GET /api/agent/capabilities
GET /api/agent/capabilities/<capability_name>
```

返回重点：

- 当前允许的能力
- 被拒绝的能力
- 参数 schema
- 当前资金和权限限制
- 是否需要人工审批

示例：

```

{
  "agent_id": "agent_strategy_assistant",
  "allow": [
    "market.read",
    "market.search",
    "market.hot_scan",
    "account.read_limited",
    "strategy.read_all",
    "strategy.detail.read",
    "strategy.workspace.read",
    "strategy.usedata.read",
    "strategy.events.read",
    "strategy.state.read",
    "strategy.draft.create",
    "strategy.draft.update",
    "strategy.batch.propose",
    "risk.check",
    "strategy.simulate",
    "strategy.submit",
    "audit.read"
  ],
  "deny": [
    "strategy.approve",
    "execution.apply",
    "admin.policy.set"
  ],
  "limits": {
    "max_strategy_budget_usdc": 100,
    "max_single_order_usdc": 20,
    "max_daily_spend_usdc": 150,
    "require_human_approval": true
  }
}

```

策略提交说明模板

GET /api/agent/capabilities 会返回 strategy_submission_template，Agent CLI 在创建或提交草案时应把该提示词放进策略生成提示里，并在 draft.agent_report 中填写结构化说明。

推荐给 Agent CLI 的提示词：

创建或提交策略草案时，请在 `draft.agent_report` 中填写结构化说明。

要求：每项 1-2 句，少写泛泛的投资观点，多写和本次市场、方向、参数直接相关的原因。

必须包含这些字段：

- `strategy_reason`：为什么选择这个市场、这个方向和这个策略代码。
- `market_observation`：当前价格/盘口/事件信息里，哪些事实支持这个草案值得人工看。
- `parameter_rationale`：`fair_price`、`entry_edge`、预算、止损止盈、时间参数等为什么这样设。
- `risk_control`：资金上限、单笔上限、退出条件和不自动实盘的边界。
- `human_review_focus`：人类批准前最需要确认的 2-3 个点。

不要写“保证盈利”“确定性机会”等结论；不确定时明确写需要人工判断。

草案示例：

```
{
  "draft": {
    "name": "Oprah 2028 nomination YES",
    "strategy_code": "Stragy_Fllow_Truth",
    "agent_report": {
      "strategy_reason": "选择该 YES 方向是因为价格很低，适合先用小额规则化草案观察，不直接代表确定性",
      "market_observation": "当前重点是核对 YES 盘口、事件截止时间和市场是否与候选人提名问题一致。",
      "parameter_rationale": "fair_price 和 entry_edge 用来限制只有明显低于主观估值时才入场，总预算",
      "risk_control": "策略默认 Stop/Virtual，批准前不自动实盘；触发止损、无成交或风控失败时优先退出",
      "human_review_focus": "请确认市场链接、方向、fair_price/entry_edge 和预算是否符合你的判断。"
    }
  }
}
```

4.2 Agent 活动状态

用途：首页同步显示 agent 正在做什么。

CLI：

```
agentctl agent.activity.current
agentctl agent.activity.list --limit 50
agentctl agent.activity.update --state AI_DRAFTING --message "正在比较 8 个热门市场"
```

REST：


```
GET /api/agent/activity/current
GET /api/agent/activity
POST /api/agent/activity
```

事件结构：

```
{
  "event_id": "evt_123",
  "agent_id": "agent_strategy_assistant",
  "state": "AI_DRAFTING",
  "message": "正在比较 8 个热门市场",
  "ref_type": "draft",
  "ref_id": "draft_123",
  "created_at": "2026-06-22T00:00:00Z"
}
```

前端首页显示：

时间	Agent	动作	关联策略/草案	状态
----	-------	----	---------	----

4.3 市场读取

用途：agent 分析你筛选的市场、行情、盘口、成交和指标。

CLI:

```
agentctl market.list --source selected --json
agentctl market.get --instrument-id xxx --json
agentctl market.orderbook --instrument-id xxx --json
agentctl market.trades --instrument-id xxx --limit 100 --json
agentctl market.metrics --instrument-id xxx --json
```

REST:

```
GET /api/agent/markets?source=selected
GET /api/agent/markets/<instrument_id>
GET /api/agent/markets/<instrument_id>/orderbook
GET /api/agent/markets/<instrument_id>/trades
GET /api/agent/markets/<instrument_id>/metrics
```

设计要求：

- 默认只允许读取用户筛选/加入自选的市场。
- 全站扫描需要单独权限。
- 返回稳定的 `instrument_id`、`condition_id`、`yes_token`、`no_token`。
- 行情字段必须标注数据时间和来源。

已实现的 Polymarket 市场查询能力：

```
agentctl market.category.list --source polymarket --json
agentctl market.search --source polymarket --category "Elections Politics,World,Geopolitics" --:
agentctl market.search --source polymarket --category "Crypto" --yes-ask-min 0.20 --yes-ask-max
agentctl market.resolve --condition-id 0x... --json
agentctl market.hot-scan --category "Elections Politics,World,Geopolitics" --sorts volume24h,vo:
agentctl strategy.batch.propose --from-hot-scan --max-drafts 5 --submit-for-approval --json
```

REST：

```
GET /api/agent/market-categories
GET /api/agent/markets
GET /api/agent/markets/resolve
POST /api/agent/market-scan
POST /api/agent/market-scan/propose-strategies
```

`/api/agent/markets` 与前端「Polymarket 市场查询」使用同一套查询能力，支持：

```
q
category / categories
sort = volume24h | volume | liquidity | spread | end_date | updated_at | price_change_24h | last
order = desc | asc
yes_ask_min / yes_ask_max
yes_bid_min / yes_bid_max
no_ask_min / no_ask_max
no_bid_min / no_bid_max
limit
refresh
```

也支持 JSON 形式传给 `POST /api/agent/market-scan`：

```
{
  "category": "Crypto",
  "sorts": "volume24h,liquidity",
  "price_filters": {
    "yes_ask": { "min": 0.20, "max": 0.45 },
    "no_bid": { "min": 0.55 }
  },
  "limit": 30
}
```

盘口范围使用当前缓存/快照里的 bid/ask；它适合发现候选市场，不代表毫秒级实时盘口触发条件。AgentMonitor 的审批详情会显示 Side Bid/Ask 和 All Bid/Ask，便于人工确认 agent 使用的价格依据。

批量扫描示例：

```
{
  "categories": ["Elections Politics", "World", "Geopolitics"],
  "sorts": ["volume24h", "volume", "liquidity", "spread"],
  "limit": 30,
  "candidate_limit": 20
}
```

批量生成待审批策略示例：

```
{
  "categories": ["Elections Politics", "World", "Geopolitics"],
  "sorts": ["volume24h", "volume", "liquidity", "spread"],
  "limit": 30,
  "candidate_limit": 20,
  "max_drafts": 5,
  "selection_mode": "yes",
  "budget_usdc": 20,
  "submit_for_approval": true
}
```

默认策略选择：

- Polymarket binary 市场默认使用 Stragy_Fllow_Truth。
- 默认选择 YES 方向；可用 selection_mode=no|cheaper|balanced 调整。
- 生成草案后自动执行 risk.check 与 strategy.simulate。

- submit_for_approval=true 时提交到 WAITING_HUMAN_CONFIRM，仍然不能由 Agent 自己批准。

4.4 账户与风险视图读取

用途：agent 了解可用预算和当前敞口，但不接触完整钱包权限。

CLI:

```
agentctl account.budget
agentctl position.list
agentctl exposure.summary
```

REST:

```
GET /api/agent/account/budget
GET /api/agent/positions
GET /api/agent/exposure
```

返回建议:

```
{
  "available_budget_usdc": 100,
  "used_budget_usdc": 35,
  "daily_spend_remaining_usdc": 65,
  "positions": [
    {
      "instrument_id": "polymarket:0x...",
      "side": "YES",
      "cost_usdc": 20,
      "avg_price": 0.39,
      "mark_price": 0.42
    }
  ]
}
```

注意：给 agent 的可以是“授权预算视图”，不一定是真实账户总余额。

4.5 策略草案

用途：agent 根据市场和用户偏好生成可审阅、可风控的策略草案。

CLI:

```
agentctl strategy.draft.create --input markets.json --json
agentctl strategy.draft.get --draft-id draft_123 --json
agentctl strategy.draft.update --draft-id draft_123 --patch patch.json --json
agentctl strategy.draft.delete --draft-id draft_123 --json
agentctl strategy.draft.list --json
```

REST:

```
POST    /api/agent/strategy-drafts
GET     /api/agent/strategy-drafts
GET     /api/agent/strategy-drafts/<draft_id>
PATCH  /api/agent/strategy-drafts/<draft_id>
DELETE  /api/agent/strategy-drafts/<draft_id>
```

草案结构:

```

{
  "draft_id": "draft_123",
  "name": "热门事件保守分批策略",
  "lifecycle_state": "AI_DRAFTING",
  "created_by": "agent_strategy_assistant",
  "thesis": "策略理由",
  "markets": [
    {
      "instrument_id": "polymarket:0x...",
      "condition_id": "0x...",
      "outcome": "YES",
      "action": "buy",
      "max_entry_price": 0.44,
      "max_exposure_usdc": 50
    }
  ],
  "budget": {
    "max_total_usdc": 100,
    "max_single_order_usdc": 20
  },
  "execution_rules": {
    "order_type": "limit",
    "cooldown_seconds": 300,
    "max_slippage_bps": 100
  },
  "exit_rules": {
    "take_profit_price": 0.62,
    "stop_loss_price": 0.30,
    "stop_new_orders_before_end_hours": 24
  },
  "risk_notes": [
    "流动性下降时暂停开仓",
    "盘口价差超过 3% 不下单"
  ]
}

```

删除规则：

- AI_DRAFTING / AI_PROPOSED 草案可以物理删除。
- WAITING_HUMAN_CONFIRM 之后不物理删除，改为 CANCELLED 或 ARCHIVED。
- 已批准或已运行的策略必须保留审计记录。

4.6 策略讨论与备注

用途：保存用户和 agent 对策略的讨论，方便复盘。

CLI:

```
agentctl strategy.comment.add --draft-id draft_123 --text "总预算降到 80"  
agentctl strategy.comment.list --draft-id draft_123
```

REST:

```
POST /api/agent/strategy-drafts/<draft_id>/comments  
GET /api/agent/strategy-drafts/<draft_id>/comments
```

备注结构:

```
{  
  "comment_id": "cmt_123",  
  "actor_type": "user",  
  "actor_id": "local_user",  
  "text": "把总预算降到 80，单笔不超过 10",  
  "created_at": "2026-06-22T00:00:00Z"  
}
```

4.7 风险检查

用途：确定性程序检查草案是否符合权限、预算、滑点、白名单和仓位限制。

CLI:

```
agentctl risk.check --draft-id draft_123 --json  
agentctl risk.explain --risk-report-id risk_123 --json
```

REST:

```
POST /api/agent/strategy-drafts/<draft_id>/risk-check  
GET /api/agent/risk-reports/<risk_report_id>
```

检查项:

- 单笔最大金额
- 单策略最大预算
- 单日最大花费
- 单市场最大敞口
- 全局最大敞口
- 最大滑点
- 市场白名单
- 是否允许该方向
- 是否允许市价单
- 市场是否 active
- 盘口价差和流动性
- 策略过期时间
- 是否需要人工审批

返回：

```
{
  "risk_report_id": "risk_123",
  "passed": false,
  "risk_level": "high",
  "violations": [
    {
      "code": "MAX_SINGLE_ORDER_EXCEEDED",
      "message": "单笔金额 50 超过上限 20",
      "field": "budget.max_single_order_usdc",
      "current": 50,
      "allowed": 20
    }
  ],
  "suggested_fixes": [
    {
      "field": "budget.max_single_order_usdc",
      "value": 20
    }
  ]
}
```

4.8 模拟与估算

用途：在审批前估算最大亏损、资金占用、成交可能性和典型场景。

CLI:

```
agentctl strategy.simulate --draft-id draft_123 --json
agentctl strategy.estimate --draft-id draft_123 --json
```

REST:

```
POST /api/agent/strategy-drafts/<draft_id>/simulate
POST /api/agent/strategy-drafts/<draft_id>/estimate
```

返回:

```
{
  "max_loss_usdc": 80,
  "max_exposure_usdc": 80,
  "estimated_orders": 4,
  "scenarios": [
    {
      "name": "take_profit",
      "estimated_pnl_usdc": 28
    },
    {
      "name": "resolve_against_position",
      "estimated_pnl_usdc": -80
    }
  ]
}
```

说明: 模拟只做场景分析, 不应包装成预测未来。

4.9 提交审批

用途: agent 把草案提交给人类确认。

CLI:

```
agentctl strategy.submit --draft-id draft_123 --json
agentctl approval.status --approval-id appr_123 --json
```

REST:

POST /api/agent/strategy-drafts/<draft_id>/submit

GET /api/agent/approvals/<approval_id>

提交时必须：

- 重新运行风险检查。
- 冻结草案版本。
- 冻结市场快照。
- 冻结预算和执行规则。
- 写入 WAITING_HUMAN_CONFIRM 。

审批单结构：

```
{
  "approval_id": "appr_123",
  "draft_id": "draft_123",
  "draft_version": 4,
  "status": "WAITING_HUMAN_CONFIRM",
  "submitted_by": "agent_strategy_assistant",
  "risk_report_id": "risk_123",
  "snapshot": {
    "markets": [],
    "budget": {},
    "execution_rules": {},
    "thesis": "..."
  },
  "expires_at": "2026-06-23T00:00:00Z"
}
```

4.10 人工审批

用途：只给人类用户、设置端或后端管理通道使用。

CLI：

```
agentctl approval.approve --approval-id appr_123 --json
```

```
agentctl approval.reject --approval-id appr_123 --reason "风险太高" --json
```

```
agentctl approval.request-changes --approval-id appr_123 --reason "总预算降到 50" --json
```

REST：

```
POST /api/approvals/<approval_id>/approve
POST /api/approvals/<approval_id>/reject
POST /api/approvals/<approval_id>/request-changes
```

权限要求：

- agent 默认不能调用。
- 审批 actor 必须是 human/admin 类型。
- 审批前再次读取最新 policy。
- 审批后生成 approved strategy 或把草案写入 strategy_registry 。

后端拒绝示例：

```
{
  "ok": false,
  "error": {
    "code": "SELF_APPROVAL_FORBIDDEN",
    "message": "Agent cannot approve its own strategy."
  }
}
```

4.11 执行计划

用途：把已批准策略转换成当前市场可执行的订单计划。

CLI：

```
agentctl execution.plan --strategy-id strat_123 --json
agentctl execution.plan.get --plan-id plan_123 --json
```

REST：

```
POST /api/execution/plans
GET /api/execution/plans/<plan_id>
```

计划要求：

- 只能引用 HUMAN_APPROVED 或 ACTIVE 策略。
- 必须引用已冻结审批快照。
- 生成计划时重新读取当前盘口。

- 如果价格、滑点、流动性不满足规则，计划失败。
- 计划必须有过期时间。

示例：

```
{
  "plan_id": "plan_123",
  "strategy_id": "strat_123",
  "orders": [
    {
      "instrument_id": "polymarket:0x...",
      "outcome": "YES",
      "side": "buy",
      "price": 0.42,
      "amount_usdc": 10,
      "order_type": "limit"
    }
  ],
  "expires_at": "2026-06-22T00:05:00Z"
}
```

4.12 执行与控制

用途：实际执行、暂停、恢复、取消策略。

CLI：

```
agentctl execution.apply --plan-id plan_123 --idempotency-key idem_123 --json
agentctl strategy.pause --strategy-id strat_123 --reason "人工暂停" --json
agentctl strategy.resume --strategy-id strat_123 --json
agentctl strategy.cancel --strategy-id strat_123 --reason "取消执行" --json
```

REST：

```
POST /api/execution/plans/<plan_id>/apply
POST /api/strategies/<strategy_id>/pause
POST /api/strategies/<strategy_id>/resume
POST /api/strategies/<strategy_id>/cancel
```

建议：

- 第一版不要给 agent execution.apply 权限。
- agent 可以提出暂停/调整建议。
- 人类确认或执行器规则通过后再执行。
- 任何执行前都要重新检查 policy 和风险。

4.13 策略监控、Action 与 PnL 读取

用途：让 agent 像人类在“策略监控 / 策略工作台”里一样读取策略运行状态、参数、UseData、Action 日志和 PnL 视图，然后用于复盘、总结和提出调整建议。

这些接口默认只读。agent 可以看、总结、建议，但不能通过这些接口改 state、平仓、删除策略或执行订单。

CLI:

```
agentctl strategy.list --limit 100 --json
agentctl strategy.get --strategy-id 53 --json
agentctl strategy.workspace --strategy-id 53 --include-events --json
agentctl strategy.usedata --strategy-id 53 --json
agentctl strategy.events --strategy-id 53 --limit 50 --json
agentctl strategy.state --strategy-id 53 --json

agentctl order.list --strategy-id strat_123 --json
agentctl trade.list --strategy-id strat_123 --json
agentctl pnl.summary --strategy-id strat_123 --json
```

REST:

```
GET /api/agent/strategies
GET /api/agent/strategies/<strategy_id>
GET /api/agent/strategies/<strategy_id>/workspace
GET /api/agent/strategies/<strategy_id>/usedata
GET /api/agent/strategies/<strategy_id>/events
GET /api/agent/strategies/<strategy_id>/state

GET /api/agent/strategies/<strategy_id>/orders
GET /api/agent/strategies/<strategy_id>/trades
GET /api/agent/strategies/<strategy_id>/pnl
```

agent 可以看到的重点：

- 策略列表：策略名、策略代码、mode、state、bankroll、exposure、PnL、更新时间、last action。
- 策略详情：leg、市场、condition_id、token、价格快照、持仓快照、editable 参数。
- 策略工作台：图表能力、市场匹配、recent events、策略运行上下文。
- UseData：策略代码本轮可读取的参数、市场、持仓、风控上下文。
- Action 日志：print/action/trade/error/settings 等事件，包含 blocked、skipped、filled、HOLD 等原因。
- State：machine/runtime/user/system state，用于判断
cooldown_active、stop_loss_locked、manual_pause_open 等运行状态。

对应设置端权限：

```
strategy_read_all      -> strategy.read_all
strategy_detail_read   -> strategy.detail.read
strategy_workspace_read -> strategy.workspace.read + strategy.usedata.read
strategy_events_read   -> strategy.events.read
strategy_state_read    -> strategy.state.read
pnl_read               -> pnl.read
```

权限边界：

- strategy_events_read 关闭时，agent 不能读取 Action 日志。
- pnl_read 关闭时，接口会隐藏 PnL / ROI / profit / fee 等收益字段。
- strategy_workspace_read 关闭时，agent 不能读取工作台和 UseData。
- 所有读取都会写入审计，且不授予 agent 任何审批或执行能力。

4.14 策略调整提案

用途：已批准或已运行策略不允许 agent 直接修改，必须提交调整提案。

CLI：

```
agentctl strategy.adjustment.propose --strategy-id strat_123 --patch patch.json --json
agentctl strategy.adjustment.submit --adjustment-id adj_123 --json
```

REST：

```
POST /api/agent/strategies/<strategy_id>/adjustments
POST /api/agent/adjustments/<adjustment_id>/submit
```

规则：

- 增加预算、扩大市场范围、提高价格上限，必须重新人工审批。
- 降低风险的调整可以配置为自动允许，例如降低单笔金额、暂停开仓。
- 原策略快照不可被覆盖，只能产生新版本或 adjustment。

4.15 知识库与报告

用途：agent 读取受控知识库、生成策略报告或复盘。

CLI：

```
agentctl knowledge.search --query "保守策略" --json
agentctl knowledge.get --doc-id doc_123 --json
agentctl artifact.create --type strategy_report --input report.md --json
```

REST：

```
GET /api/agent/knowledge/search
GET /api/agent/knowledge/<doc_id>
POST /api/agent/artifacts
```

设计要求：

- 不开放全盘文件读取。
- 只开放指定知识库目录或数据库。
- artifact 要绑定 draft/strategy/approval。

4.16 任务与调度

用途：agent 触发低风险后台任务，例如刷新行情、生成报告。

CLI：

```
agentctl job.run --name refresh_markets --json
agentctl job.status --job-id job_123 --json
agentctl job.logs --job-id job_123 --tail 100 --json
```

REST：

```
POST /api/agent/jobs
GET /api/agent/jobs/<job_id>
GET /api/agent/jobs/<job_id>/logs
```

要求：

- job 必须白名单。
- 不允许 agent 提交任意 shell。
- 每个 job 有超时、日志和失败原因。

4.17 通知

用途：agent 把需要人类处理的事情推到首页、通知栏或审批队列。

CLI：

```
agentctl notify.send --template strategy_ready --data data.json --json
```

REST：

```
POST /api/agent/notifications
```

常见通知：

- 新策略等待确认
- 风控失败
- 策略需要修改
- 执行计划失效
- 订单失败
- 敞口接近上限

4.18 审计

用途：记录 agent、用户、系统所有关键动作。

CLI：

```
agentctl audit.list --actor agent_strategy_assistant --limit 100 --json
agentctl audit.get --event-id audit_123 --json
```


REST:

```
GET /api/audit
GET /api/audit/<event_id>
```

审计字段:

```
{
  "event_id": "audit_123",
  "run_id": "run_123",
  "step_id": "step_123",
  "workflow_id": "eventgraph_investigation_20260629",
  "agent_kind": "external",
  "actor_type": "agent",
  "actor_id": "agent_strategy_assistant",
  "capability": "strategy.draft.update",
  "target_type": "draft",
  "target_id": "draft_123",
  "endpoint": "/api/agent/strategy-drafts/draft_123",
  "method": "PATCH",
  "status_code": 200,
  "duration_ms": 184,
  "input_summary": {},
  "output_summary": {},
  "error_json": null,
  "policy_decision": "allow",
  "risk_decision": "not_required",
  "created_at": "2026-06-22T00:00:00Z"
}
```

审计日志不能被 agent 删除。

4.19 Agent Run / Step 调用链

为方便 debug，AgentMonitor 不只看最终审计事件，还应能读取一次 agent 调用背后的完整步骤：

```
agent_runs
-> agent_run_steps
-> agent_audit_events
```

REST:

```
GET /api/agent/runs
GET /api/agent/runs/<run_id>/steps
GET /api/agent/audit?run_id=run_123
GET /api/agent/audit?agent_kind=internal|external|human|system
```

建议外接 agent 每次任务都显式传入同一个 run_id 和 workflow_id，多步任务再为每步传入不同 step_id。如果没有传入，后端会为单次 API 调用生成最小 run/step，保证后台仍可追踪。

关键字段：

run_id	一次 agent 任务或工作流
step_id	任务里的单个工具/API/LLM 步骤
workflow_id	业务流程标识，例如 eventgraph_investigation
agent_kind	internal / external / human / system
endpoint	被调用的 HTTP 路径
method	HTTP 方法
status_code	成功或失败状态码
duration_ms	步骤耗时
error_json	失败时的结构化错误

/api/agent/* 和 /api/approvals/* 的失败请求也应进入 run/step/audit，避免只在前端或 Flask 日志里看到错误。

5. 权限与设置端

5.1 权限模型

设置端管理 agent_policy：

```

{
  "agent_id": "agent_strategy_assistant",
  "enabled": true,
  "role": "strategy_assistant",
  "allow": [
    "market.read",
    "account.read_limited",
    "strategy.draft.create",
    "strategy.draft.update",
    "strategy.draft.delete",
    "risk.check",
    "strategy.simulate",
    "strategy.submit",
    "order.read",
    "pnl.read"
  ],
  "deny": [
    "strategy.approve",
    "execution.apply",
    "admin.policy.set"
  ],
  "limits": {
    "max_strategy_budget_usdc": 100,
    "max_single_order_usdc": 20,
    "max_daily_spend_usdc": 150,
    "max_market_exposure_usdc": 50,
    "max_global_exposure_usdc": 300,
    "max_slippage_bps": 100,
    "allowed_market_ids": [],
    "allowed_venues": ["polymarket"],
    "allow_market_order": false,
    "require_human_approval": true,
    "approval_expires_minutes": 1440,
    "expires_at": "2026-07-01T00:00:00Z"
  }
}

```

5.2 必须锁死的权限

这些限制必须在后端执行层检查：

- agent 不能批准自己的策略。

- agent 不能提高自己的预算。
- agent 不能修改自己的权限。
- agent 不能修改钱包、密钥、交易凭证。
- agent 不能执行未批准策略。
- agent 不能绕过 `risk.check` 。
- agent 不能把策略从 `WAITING_HUMAN_CONFIRM` 直接改成 `HUMAN_APPROVED` 。
- agent 不能物理删除已提交、已批准、已运行策略。

5.3 设置端功能

设置端建议提供：

Agent 启用/禁用
能力勾选
查看全部策略列表
查看策略详情
查看策略工作台 / UseData
查看策略 Action 日志
查看策略 State
单笔最大金额
单策略最大金额
每日最大花费
单市场最大敞口
全局最大敞口
最大滑点
允许市场白名单
允许交易方向
是否允许市价单
是否允许自动执行
审批阈值
权限过期时间
紧急停止
审计日志查看

6. AgentMonitor 工作台设计

AgentMonitor 建议拆成四个视图，分别服务于总览、内置 agent 调查、外接 agent 策略协作和完整审计回放。

总览
内置 Agent
外接 Agent
审计日志

6.1 总览

总览聚合最需要人类快速判断的状态：

待确认策略
策略草案
内置 EventGraph 调查
风控阻断计数
最近失败步骤

总览只做摘要，不替代完整审计。

6.2 内置 Agent

内置 Agent 专门显示系统内部产生的调查、提案和辅助分析记录。

纳入范围：

```
actor_type = internal_agent  
agent_kind = internal  
target_type = event_investigation  
target_type = event_proposal  
capability 以 event.investigation / event.proposal 开头
```

第一版重点是 EventGraph investigation/proposal 类记录。内置 agent 可以读取新闻、市场和图谱，生成调查报告或 proposal，但不直接覆盖正式 EventGraph。

6.3 外接 Agent

外接 Agent 保留策略审批、策略草案和最近活动。

Agent Activity

显示 agent 当前和最近动作：

时间 | Agent | 当前动作 | 关联对象 | 状态

示例：

12:01 | Strategy Agent | 正在分析 8 个热门市场 | draft_123 | AI_DRAFTING
12:03 | Strategy Agent | 已完成风险检查 | risk_123 | AI_PROPOSED
12:05 | Strategy Agent | 已提交审批 | appr_123 | WAITING_HUMAN_CONFIRM

Pending Human Confirm

显示所有 WAITING_HUMAN_CONFIRM 的审批单。

每条展示：

策略名
Agent 理由摘要
市场数量
最大预算
最大单笔
最大亏损估算
风险等级
提交时间
过期时间

操作：

查看详情
批准
拒绝
要求修改
归档

批准前必须展示：

- 冻结的市场快照
- 资金参数
- 风控报告
- 模拟结果
- 与当前行情差异

Strategy Monitor

现有策略监控首页可以补充：

AI State / Approval State
Created By
Approval
Risk
Agent Last Action

行级操作建议：

状态	可见操作
AI_DRAFTING	查看、编辑、删除草案
AI_PROPOSED	查看、编辑、提交审批、删除草案
WAITING_HUMAN_CONFIRM	查看、批准、拒绝、要求修改
HUMAN_APPROVED	生成执行计划、转 Virtual、归档
ACTIVE	暂停、查看订单、查看 PnL、提出调整
PAUSED	恢复、取消、归档
CANCELLED / ARCHIVED	只读查看

6.4 审计日志

审计日志保留完整流水，支持：

搜索
类别筛选
agent_kind 筛选
run_id 筛选
清除当前筛选
查看失败 error_json

审计日志不是只给 UI 看，也应作为后端 debug 的事实来源。

6.5 修改与删除语义

前端按钮语义：

删除草案 = 只适用于未提交 `draft`

取消策略 = 已提交/已批准/运行中的策略不再执行

归档策略 = 历史只读隐藏

要求修改 = 回到 `HUMAN_REVISION_REQUESTED`

不要对已审批策略做静默覆盖。

7. 数据库建议

第一版可以新增这些表，或按现有服务拆分成等价结构：

```
agent_policies
agent_activity_events
agent_runs
agent_run_steps
strategy_drafts
strategy_draft_versions
strategy_approval_requests
strategy_approval_snapshots
strategy_adjustment_requests
agent_audit_events
```

7.1 strategy_drafts

```
draft_id
name
lifecycle_state
created_by_type
created_by_id
current_version
draft_json
risk_report_id
created_at_utc
updated_at_utc
```


7.2 strategy_draft_versions

version_id
draft_id
version_number
draft_json
change_reason
created_by_type
created_by_id
created_at_utc

7.3 strategy_approval_requests

approval_id
draft_id
draft_version
status
submitted_by_type
submitted_by_id
approved_by_type
approved_by_id
risk_report_id
expires_at_utc
created_at_utc
updated_at_utc

7.4 strategy_approval_snapshots

approval_id
snapshot_json
market_snapshot_json
policy_snapshot_json
risk_snapshot_json
created_at_utc

审批通过后，再把冻结快照转换为正式策略：

```
strategy_registry  
strategy_legs  
strategy_state
```

这样不会污染当前正式策略表，也能保留清晰审批历史。

7.5 agent_runs

```
run_id  
actor_type  
actor_id  
agent_kind  
workflow_id  
capability  
target_type  
target_id  
status  
started_at_utc  
finished_at_utc  
input_json  
output_json  
error_json
```

7.6 agent_run_steps

```
step_id
run_id
step_order
actor_type
actor_id
agent_kind
capability
target_type
target_id
endpoint
method
status
status_code
duration_ms
started_at_utc
finished_at_utc
input_json
output_json
error_json
```

7.7 agent_audit_events

除原有审计字段外，需要补充调用链字段：

```
run_id
step_id
agent_kind
workflow_id
endpoint
method
status_code
duration_ms
error_json
```

这样 AgentMonitor 可以从审计事件跳回某次 run，也可以从 run 展开所有步骤。

8. MVP 范围

建议第一阶段只做：

capabilities
agent.activity.current/list
market.list/get/orderbook
account.budget
position.list
strategy.draft.create/get/update/delete/list
risk.check
strategy.simulate
strategy.submit
approval.approve/reject/request-changes（人类专用）
audit.list
agent.runs
agent.run_steps
首页 Pending Human Confirm
首页 Agent Activity
AgentMonitor 内置 Agent / 外接 Agent / 审计日志分栏

暂缓：

execution.apply 给 agent 调用
自动实盘执行
复杂调度
全站市场扫描
多 agent 自主协作
自动提高预算

推荐第一阶段工作流：

用户筛选市场

- > agent 读取 selected markets
- > agent 生成 draft
- > agent 和用户讨论并修改 draft
- > risk.check
- > simulate
- > agent submit
- > WAITING_HUMAN_CONFIRM
- > 首页人工批准/拒绝/要求修改
- > 批准后进入 Virtual 或生成执行计划

9. 后端强制校验清单

每次 agent 调用写接口前，后端必须检查：

- agent 是否启用
- capability 是否允许
- deny 是否命中
- policy 是否过期
- 目标市场是否允许
- 金额是否超过单笔/单策略/每日/单市场/全局限制
- 是否需要人工审批
- 是否 self-approval
- 是否引用最新草案版本
- 是否已过期
- 是否有 idempotency_key
- 是否写入 audit

最小错误码集合：

AGENT_DISABLED
CAPABILITY_DENIED
POLICY_EXPIRED
LIMIT_EXCEEDED
MARKET_NOT_ALLOWED
RISK_CHECK_FAILED
HUMAN_APPROVAL_REQUIRED
SELF_APPROVAL_FORBIDDEN
STALE_DRAFT_VERSION
APPROVAL_EXPIRED
IDEMPOTENCY_CONFLICT

10. Agent CLI 端到端案例

本节用一个完整例子说明：agent 通过受控 CLI 连接系统后，能看到什么、怎样分析、怎样提交策略，以及哪些动作必须交给人类。

说明：agentctl 是建议封装的受控 CLI，底层对应 /api/agent/*。如果暂时没有独立 CLI，也可以用 PowerShell / curl 直接调用同一批 REST 接口。

10.1 场景

人类给 agent 的任务：

请查看今天策略运行情况，并扫描 Elections Politics、World、Geopolitics 中热门且市值/流动性较大的 Pol

系统边界：

```
max_strategy_budget_usdc = 100
max_single_order_usdc = 20
proposal_budget_usdc = 20
proposal_single_order_usdc = 5
require_human_approval = true
deny = strategy.approve, execution.apply, admin.policy.set
```

10.2 第一步：agent 读取自己有哪些权限

CLI:

```
agentctl capabilities --json
```

REST:

```
Invoke-RestMethod -Uri "http://127.0.0.1:5001/api/agent/capabilities"
```

agent 应该先检查:

- 是否启用 agent 接口。
- 是否允许 market.search / market.hot_scan 。
- 是否允许 strategy.read_all / strategy.events.read / strategy.state.read 。
- 单笔、单策略、每日、全局敞口上限是多少。
- 是否必须人工审批。

如果缺少必要权限，agent 只能报告“权限不足”，不能绕过设置端。

10.3 第二步：agent 读取今天策略运行情况

CLI:

```
agentctl strategy.list --limit 100 --json  
agentctl strategy.events --strategy-id 53 --limit 20 --json  
agentctl strategy.state --strategy-id 53 --json
```

REST:

```
Invoke-RestMethod -Uri "http://127.0.0.1:5001/api/agent/strategies?limit=100"  
Invoke-RestMethod -Uri "http://127.0.0.1:5001/api/agent/strategies/53/events?limit=20"  
Invoke-RestMethod -Uri "http://127.0.0.1:5001/api/agent/strategies/53/state"
```

agent 能看到类似信息:

```
{
  "strategy_id": 53,
  "strategy_name": "Gavin Newsom 2028 Democratic Nomination YES",
  "strategy_code": "Stragy_Fllow_Truth",
  "mode": "Virtual",
  "state": "auto",
  "strategy_bankroll": 100,
  "exposure": 35.87,
  "strategy_pnl": -3.20,
  "last_action": "[DECISION] HOLD, signal=add_edge"
}
```

agent 结合 events / state 后可以给出摘要：

今天共有 15 个策略，14 个 auto，1 个 stop_loss_locked。

组合总 PnL 约 -40.32 USDC，主要亏损来自 Strait of Hormuz 策略的历史止损锁定。

Gavin 策略目前是 HOLD，cooldown 未拦截，但 MACD/仓位/edge 组合没有触发实际加仓。

多个 Agent Hot Scan 策略被 MACD flat 阻止继续加仓，说明系统偏保守。

注意：如果 pnl_read=false，上述摘要不能包含 PnL、ROI、profit、fee 等字段。

10.4 第三步：agent 扫描热门市场

CLI:

```
agentctl market.hot-scan \
  --categories "Elections Politics,World,Geopolitics" \
  --sorts "volume24h,volume,liquidity,spread" \
  --limit 30 \
  --candidate-limit 20 \
  --json
```

REST:


```
$body = @{
    categories = @("Elections Politics", "World", "Geopolitics")
    sorts = @("volume24h", "volume", "liquidity", "spread")
    limit = 30
    candidate_limit = 20
} | ConvertTo-Json
```

```
Invoke-RestMethod `
    -Method Post `
    -Uri "http://127.0.0.1:5001/api/agent/market-scan" `
    -ContentType "application/json" `
    -Body $body
```

agent 能看到的市场字段包括：

```
question
category
url
condition_id
yes/no token id
yes_price / no_price
best_bid / best_ask
spread
liquidity
volume / volume24h
end_date
active / accepting_orders
```

agent 不应该只按一个维度选择。推荐综合：

- 成交量和流动性足够。
- spread 不过大。
- 市场 active 且 accepting_orders。
- 到期时间与策略节奏匹配。
- 问题规则清楚，不容易因歧义误判。
- 当前价格与 agent 估计 fair_price 有可解释差距。

10.5 第四步：agent 批量生成策略草案

CLI:

```
agentctl strategy.batch.propose \
  --from-hot-scan \
  --max-drafts 5 \
  --selection-mode yes \
  --budget-usdc 20 \
  --single-order-usdc 5 \
  --submit-for-approval \
  --json
```

REST:

```
$body = @{
  categories = @("Elections Politics", "World", "Geopolitics")
  sorts = @("volume24h", "volume", "liquidity", "spread")
  limit = 30
  candidate_limit = 20
  max_drafts = 5
  selection_mode = "yes"
  budget_usdc = 20
  single_order_usdc = 5
  submit_for_approval = $true
} | ConvertTo-Json
```

```
Invoke-RestMethod `
  -Method Post `
  -Uri "http://127.0.0.1:5001/api/agent/market-scan/propose-strategies" `
  -ContentType "application/json" `
  -Body $body
```

每个草案必须包含模板化说明，方便人类审批：

```
{
  "agent_report": {
    "strategy_reason": "为什么选择该市场、方向和策略代码。",
    "market_observation": "当前价格、盘口、成交量、流动性和规则里哪些事实值得关注。",
    "parameter_rationale": "fair_price、entry_edge、预算、止损止盈、时间参数为什么这样设置。",
    "risk_control": "资金上限、单笔上限、退出条件、不自动实盘的边界。",
    "human_review_focus": "人类批准前最需要确认的 2-3 个点。"
  }
}
```

10.6 第五步：风控、模拟和提交审批

如果是单个草案流程，agent 应按顺序调用：

```
agentctl risk.check --draft-id draft_123 --json
agentctl strategy.simulate --draft-id draft_123 --json
agentctl strategy.submit --draft-id draft_123 --json
```

对应 REST：

```
POST /api/agent/strategy-drafts/<draft_id>/risk-check
POST /api/agent/strategy-drafts/<draft_id>/simulate
POST /api/agent/strategy-drafts/<draft_id>/submit
```

提交成功后状态应进入：

WAITING_HUMAN_CONFIRM

首页 Agent 工作台 会同步看到：

待人工确认：新增审批单

Agent Activity: 完成风控 / 完成模拟 / 提交人工确认

策略草案：保留草案版本与参数快照

如果风控失败，agent 不能强行提交，只能：

- 降低预算或单笔金额。
- 修改 max_entry_price / entry_edge 等参数。
- 重新跑 risk.check。
- 或向人类报告风险失败原因。

10.7 第六步：人类审批与人工超控

人类在首页或审批弹窗中查看：

市场快照

完整参数

agent_report

风险检查

模拟结果

预算和单笔限制

Action / audit 记录

人类可以：

批准

拒绝

要求修改

手动修改参数后再批准

规则：

- agent 不能批准自己提交的策略。
- 人类手动修改参数后，可以作为人工超控重新审批。
- 后端审批时仍会重跑或读取最新风控结果；风险不通过时默认禁止批准，除非后续专门设计“人工带理由强制批准”的高权限通道。

10.8 第七步：审计和复盘

CLI：

```
agentctl audit.list --actor agent_strategy_assistant --limit 100 --json
```

```
agentctl agent.activity.list --limit 50 --json
```

REST：

```
Invoke-RestMethod -Uri "http://127.0.0.1:5001/api/agent/audit?limit=100"
```

```
Invoke-RestMethod -Uri "http://127.0.0.1:5001/api/agent/activity?limit=50"
```

审计里应能追踪：

agent 何时读了哪些策略
agent 何时扫描了哪些市场
agent 创建/修改了哪个 draft
风控结果是什么
模拟结果是什么
何时提交给人类
人类何时批准/拒绝/要求修改

10.9 本案例的权限闭环

这个流程里，agent 的能力边界是：

阶段	agent 能做	agent 不能做
观察策略	读列表、详情、工作台、UseData、Action、State	改 state、删除策略、平仓
扫描市场	按分类、排序、流动性读取候选	绕过市场白名单
生成草案	创建/修改/删除未提交草案	提高预算上限
风控模拟	运行 risk.check / simulate	忽略风控失败
提交审批	进入 WAITING_HUMAN_CONFIRM	自己批准
审批后	查看结果、复盘、提出调整建议	直接实盘执行、修改权限

因此，agent 可以完成“研究、总结、草案、风控、提交”的闭环；资金最终确认、权限配置和执行边界仍由人类与后端风控掌握。

11. 结论

该设计把 agent 定位为策略助理：

agent 负责研究、草案、解释、提交
系统负责风控、冻结、执行、审计
人类负责确认、资金、权限、开关

这样既能让 agent 真正帮忙写策略，又不会让它绕过人的最终确认和资金边界。